

Gofer v4: A Low-Budget, Resumable Self-Play Training Pipeline for 9×9 Go, Built on the KataGo Recipe

DevomB
Gofer project

September 2026

Abstract

KataGo [51] showed that a handful of changes to the AlphaZero process—playout cap randomization, global pooling, auxiliary policy, ownership and score targets, and a growing replay window—cut the compute needed to learn Go by about $50\times$. Even so, that result needed tens of GPUs. This report describes version 4 of the training pipeline for Gofer, a small open-source Go engine. The goal is different: build the infrastructure so a KataGo-style run can start on free CPU runners and move to a single rented, preemptible GPU without losing work.

Building the gate turned out to be the most valuable part. Asked to decide whether one network was stronger than another, it could not: the answer was exactly 0.500 every time. Chasing that exposed eight defects, all older than this work and all invisible to the test suite. Five were in the search. On 9×9 it never left the root at all, because a stale pointer lost the flag that marks a node expanded; with that repaired, every playout went down a single line, because first-play urgency was a flat constant no visited child could lose to; selection maximised the *opponent's* value, so more search made the engine weaker; two passes were never treated as the end of the game; and forced playouts, applied to every child rather than every *visited* child, outnumbered the search budget and flattened the policy training target. The other three were in the measurement apparatus, and they are why the first five survived: the arena drew correlated openings for one of the two roles, the reproducible strength baseline silently enhanced both players in a way that bypassed the root, and matches stopped early on the very statistic they were measuring. The project's own reproducible strength baseline—200 games, identical evaluators, one side searching three times harder—had recorded the deeper search winning **7 games in 200**, or -576 Elo. The same command now returns **149 in 200**, $+186$ Elo, with the median game growing from 11 moves to 76. Every strength number the project had recorded was measuring these bugs, and the reference baseline asserted that more search was catastrophically harmful. A later pass over the engine, made for code quality rather than correctness, found two more of the same kind by reading alone: a sidecar-backed gate that pointed both engines at the same model and so returned the 0.500 a working symmetric match would, and a transposition table that stored no key and served colliding positions each other's evaluations.

We describe the full loop. Go self-play writes compact binary shards. A new learner masks the policy loss to full-search positions, splits train and validation by game, augments with the eight board symmetries, keeps an exponential moving average of the weights, and can distill a new architecture from the current champion. An orchestrator adds sequential (SPRT) gating, stage-level resumability, and a registry of champions that is never pruned.

Relative to the previous Gofer pipeline, we fix two labeling defects and a validation leak, and make the data about $40\times$ smaller. The loop was verified end to end on CPU, with a checked parity bound between PyTorch and ONNX Runtime. We report no playing-strength results: no training run at scale has been done. We state precisely which KataGo techniques we adopt, which we change, and which are still missing, and we give the experiment protocol that would test the claims left open.

1 Introduction

AlphaGo Zero [42] and AlphaZero [43] learned superhuman Go from self-play alone. They did it with enormous compute: about 41 TPU-years for AlphaZero’s main Go run, and about 74 GPU-years for the ELF OpenGo replication [47], by the estimates of Wu [51]. Wu’s KataGo paper, the reference point throughout this report, closed much of that gap. It surpassed ELF’s final model after 19 days on fewer than 30 V100 GPUs, a reduction of about $50\times$. It got there through a set of mostly general techniques: playout cap randomization, policy target pruning, global pooling, an auxiliary opponent-policy target, and auxiliary ownership and score targets.

Gofer is a small Go engine written in Go. It uses Chinese area scoring, MCTS with a PUCT selection rule, a GTP interface, and ONNX neural evaluation. Its README cites KataGo as its inspiration. Until this work, its learning loop (“v3”) was a shell script. The script generated self-play JSONL on a CPU cloud instance, fine-tuned a 6-block, 64-channel residual network, and promoted a candidate after an arena of up to 200 games against the champion. Two constraints shape this report:

1. **No compute budget.** The owner cannot rent a GPU today. The pipeline therefore has to do something useful on free CPU resources, and it has to make a future rented GPU, most likely a cheap preemptible instance, as productive as possible from the first hour.
2. **No strength results yet.** Without a real training run, we cannot honestly claim that the v4 pipeline produces a stronger engine than v3, let alone one comparable to KataGo. Every claim below is limited to what we measured or can derive, and section 9 lists what remains untested.

What “better” means here. The v4 pipeline improves on its predecessor in four concrete ways, and it differs from the KataGo paper’s recipe in several deliberate ways:

- **Correct targets.** v3 stored ownership in absolute colour while every other target is from the side to move. Its `policy_opp` field held the *previous* move’s policy rather than the opponent’s reply. It split validation by row, so near-identical positions from one game landed on both sides of the split. All three are fixed, and tests cover each fix (sections 4 and 5).
- **Adoption of KataGo’s general techniques at small scale.** We adopt playout-cap-aware training, global-pooling bias blocks, the auxiliary opponent-policy head, ownership and score heads, and the power-law replay window (sections 5 and 6). One change is deliberate: KataGo *discards* fast-search positions, while we keep them and mask only the policy loss. Section 5.2 discusses the correlation risk this brings.
- **Cheaper iteration.** The data is about $40\times$ smaller, with no JSON parsing (section 4), and self-play for the next cycle overlaps training.
- **Controlled promotion errors.** Gating uses a sequential probability ratio test [49]. It stops early on clearly worse or clearly better candidates, and it cuts the false-promotion rate at zero true gain from 8.0% for v3 to 2.4%, while promoting a +50 Elo candidate 90% of the time against v3’s 67%. The price is *longer* gates near zero gain (section 6), so this is an error-control gain, not a cost saving.
- **Eight defects, found by building the gate.** Five in the search: a stale pointer that stopped it leaving the root on 9×9 ; a first-play-urgency constant that collapsed the visit

distribution—the policy training target—onto one move; forced playouts that outnumbered the search budget and flattened the same target; an inverted value sign that made more search strictly weaker; and a search that never saw a game end. Three in the measurement apparatus, which is why the others survived: correlated per-game seeds, a strength baseline that silently enhanced both players, and matches that stopped on the statistic under test. On the project’s own reproducible baseline, the side searching three times harder went from 7 wins in 200 games to 149, a sign flip from -576 to $+186$ Elo (section 8.5). This is the strongest argument in this report for building the measurement apparatus before spending money on compute.

- **Robustness to cheap hardware.** Every stage can resume after a crash or preemption, the trainer can continue mid-run, and champions are immutable and never pruned (sections 6 and 7). A distillation path lets the project change network architecture without discarding the champion’s knowledge (section 5.5). That cost was the reason an earlier ablation kept the 6×64 network.

Contributions. (i) A complete, tested description of a small-scale KataGo-style pipeline for 9×9 Go, from Go self-play to a gated, published ONNX champion. (ii) An explicit technique-by-technique comparison with Wu [51] (table 1). (iii) Measurements of what can be measured without a training run: data size, load time, parameter and compute budgets, CPU inference latency, export parity, exact gate operating characteristics, and end-to-end cycle timings (section 8), together with the engine defects that apparatus exposed (section 8.5). (iv) An ablation protocol, modelled on KataGo’s, for the experiments that would settle the open questions (section 10).

Organisation. Section 2 reviews AlphaZero-style training and the relevant parts of KataGo. Section 3 gives the system overview and comparison tables. Sections 4 to 7 describe self-play data, the learner, orchestration, and infrastructure in turn. Section 8 reports measurements, section 9 lists limitations, and section 10 concludes.

2 Background

2.1 AlphaZero-style self-play

Monte Carlo tree search [5, 26] guided by a policy and value network is the basis of AlphaGo [41], AlphaGo Zero [42], and AlphaZero [43], together with their open reproductions ELF OpenGo [47], Leela Zero [36] and SAI [33].

Each playout descends from the root. At each node it picks the child c that maximises the PUCT score [40, 42]:

$$\text{PUCT}(c) = V(c) + c_{\text{PUCT}} P(c) \frac{\sqrt{\sum_{c'} N(c')}}{1 + N(c)}, \quad (1)$$

where V is the mean value of the subtree, P the network’s prior and N the visit count. The root visit distribution π becomes the policy target, the game outcome z the value target, and the network is trained to minimise

$$L = (z - \hat{v})^2 - \sum_m \pi(m) \log \hat{\pi}(m) + c \|\theta\|^2. \quad (2)$$

AlphaGo Zero also exploited the eight rotations and reflections of the board to augment its training data [42, 43], and it gated each new network by requiring a 55% win rate against the current one over 400 games [42]. AlphaZero dropped gating and trained continuously [43].

2.2 The KataGo recipe

Wu [51] keeps the AlphaZero skeleton and changes the parts listed below. This report refers back to them by name.

Playout cap randomization. Only a fraction $p = 0.25$ of turns gets a full search (a cap of N nodes); the rest use a fast search with cap $n < N$. Early in the run $(N, n) = (600, 100)$, rising to $(1000, 200)$. *Only full-search turns are recorded for training.* Fast turns exist to generate more games, and therefore more value targets, per unit of compute. Informal evidence from Leela Zero suggests that policy learning needs far more playouts per move than value learning [11].

Forced playouts and policy target pruning. Children of the root get a minimum number of playouts, $n_{\text{forced}}(c) = (kP(c) \sum N)^{1/2}$ with $k = 2$. The forced extras are then subtracted from the policy target, so exploration no longer pollutes what the policy learns.

Global pooling. Pooling layers compute each channel’s mean, its mean scaled by board width, and its maximum. A fully connected layer maps these to a per-channel bias for another set of channels. The structure follows the first convolution of some residual blocks and appears in the policy and value heads. It is related to Squeeze-and-Excitation [20].

Auxiliary policy target. The policy head also predicts the opponent’s reply, with weight $w_{\text{opp}} = 0.15$ [46].

Ownership and score targets. The network predicts the final owner of each point (cross-entropy with $w_o = 1.5/b^2$) and the final score, via a “pdf” and a “cdf” loss each with weight 0.02. This follows the multi-labelled value networks of Wu et al. [52].

Game-specific input features. Liberties, ladders, pass-alive regions and komi parity.

Training details. SGD with momentum 0.9, batch 256, $c_{L2} = 3 \times 10^{-5}$, value-loss scale $c_g = 1.5$. The per-sample learning rate is fixed, lowered for the first 5M samples and near the end. Samples are drawn uniformly from a moving window whose size grows sublinearly with the total amount of data:

$$N_{\text{window}} = c \left(1 + \beta \frac{(N_{\text{total}}/c)^\alpha - 1}{\alpha} \right), \quad c = 250,000, \quad \alpha = 0.75, \quad \beta = 0.4 \quad (3)$$

[51, App. C]. Candidates are an exponential moving average of weight snapshots, in the spirit of stochastic weight averaging [23]. A candidate must win at least 100 of 200 games to replace the current network [51, App. E].

Wu’s ablations (Table 2 of that paper) put the training-time factor of each technique between $1.25\times$ and $1.65\times$ at about two days of compute. The factors multiply to roughly $9\times$. This report uses those factors only as motivation. We do not reproduce them.

2.3 Gofer before this work

Gofer plays under Chinese area scoring with Tromp–Taylor-style evaluation [48]. Its network interface is fixed by a feature schema:

- **Spatial input** ($8 \times S \times S$): own stones, opponent stones, empty points, the simple-ko point, a to-move plane, and the last three moves.

- **Global input** (4 values): komi/10, the normalised move number, and one-hot side to move.
- **Outputs:** $S^2 + 1$ policy logits (pass last) and a scalar tanh value from the side to move’s perspective.

Search uses eq. (1) with $c_{\text{UCT}} = 1.1$. Unvisited children get a fixed value of -0.2 , whereas KataGo’s first-play urgency is the parent value minus $0.2\sqrt{P_{\text{explored}}}$. In self-play, root Dirichlet noise ($\alpha = 0.15$, blend 0.25) is applied on every move, fast and full, whereas KataGo disables it on fast searches. Full searches also force a minimum number of visits on each root child ($2 + \lfloor \sqrt{P(c)(N+1)} \rfloor$) for playout cap N , versus KataGo’s $\sqrt{2P(c)\sum N}$, and they prune the policy target by dropping children with fewer than two visits (section 4.1). Inference runs through ONNX Runtime [35], either in-process or through a Python sidecar.

The v3 loop, documented in the repository’s ADR 0003, ran the following cycle:

1. Self-play: 200 games per cycle, 200 playouts per move with playout cap randomization ($p = 0.20$, fast cap 50), keeping only full-search rows.
2. Replay: append the games to a JSONL replay file capped first-in-first-out at 50,000 rows.
3. Training: fine-tune the champion’s 6-block, 64-channel network (“6×64”) for 15 epochs with SGD.
4. Gating: export the candidate to ONNX and play up to 200 arena games against the champion. Promote if it scores at least 0.55 with the lower Wilson bound [50] above 0.5. The arena stops early: after 20 games once 0.55 is out of reach, and after 100 games once the promotion condition holds.

ADR 0005 compared net sizes offline and found a thin validation-loss edge for a 4×48 network. It kept 6×64 anyway, because changing architecture breaks warm-starting and was estimated to cost 4–5 days and about 20 cycles to recover the champion’s strength on the project’s CPU instance. Section 5.5 targets exactly that cost.

3 System Overview

Figure 1 shows one v4 cycle. Generation k is the current champion, stored as immutable ONNX, PyTorch and checkpoint files. It plays self-play games in the Go engine, which writes one compressed shard per cycle (section 4). The learner warm-starts from generation k ’s weights and trains on a window over all shards (section 5). It then exports a candidate, and that export must pass a numeric parity check. The orchestrator gates the candidate against generation k with a sequential test. If it passes, the candidate becomes generation $k+1$ and is published to a champion registry (section 6). Meanwhile, self-play for the next cycle is already running.

Interfaces. The three components meet at two narrow contracts. That let two developers (here, two agent sessions) build them in parallel and test them separately.

1. **Data.** GOFER SHARD v1 is a `.npz` archive with a fixed set of typed arrays (table 2 in section 4). The Go engine writes it, and the learner’s packer can also produce it from legacy JSONL.

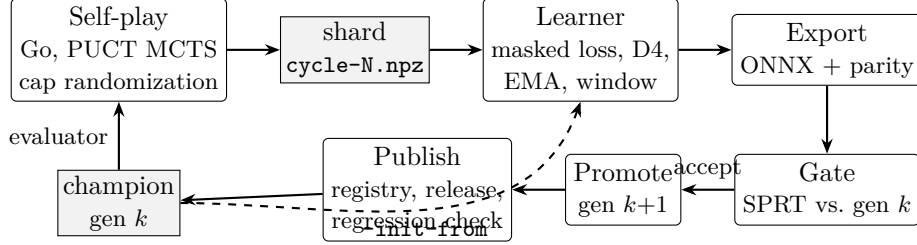


Figure 1: One v4 training cycle. Every arrow is a resumable stage recorded in the run’s state file. Self-play for cycle $N+1$ starts while cycle N trains and gates.

2. **Learner command line.** The trainer takes `-data` (files or directories), `-window-rows`, `-init-from` or `-continue`, and an optional `-config` preset. It writes `best.pt`, a self-describing `best.ckpt`, a full-state `last.pt`, and a `summary.json` with fixed keys. The exporter takes a checkpoint and writes an ONNX file whose input and output names match the engine’s schema, exiting non-zero if parity fails.

Comparison. Table 1 places v4 next to the v3 loop and the published KataGo recipe [51], one technique per row. Four symbols are used: $\checkmark$ means adopted as published, $\sim$ means adopted with a stated change, $\times$ means absent, and “new” means not in the KataGo paper.

The table is deliberately balanced. On the core learning techniques, v4 is a *subset* of KataGo with a few simplifications. Policy target pruning is only a visit-threshold approximation, and the score-belief distribution, Go-specific features and rules randomization are all missing. The additions are mostly engineering: data format, resumability, sequential gating, distillation. These matter when compute is scarce and preemptible. They do not change what the network can learn in principle.

4 Self-Play and the Training Data

Self-play runs in the Go engine, which is written in Go [8]. This section covers how a game is played, what each position is labelled with, and the binary *shard* format that replaces JSONL between the engine and the learner.

4.1 Search during self-play

Each move is chosen by Monte Carlo tree search with the PUCT selection rule of eq. (1) [40, 42], using $c_{\text{PUCT}} = 1.1$ as in Wu [51]. Two details differ from KataGo. An unvisited child is given the fixed value -0.2 , whereas KataGo’s first-play urgency is the parent’s value minus $0.2\sqrt{P_{\text{explored}}}$. And the constant 1.03, which KataGo and SAI [33] use as a softmax temperature on the root policy, is applied in Gofer as a divisor on the root children’s mean values. Root priors are mixed with Dirichlet noise ($\alpha = 0.15$, weight 0.25) on *every* self-play move. KataGo instead turns noise and other exploration off on fast searches to keep them as strong as possible [51, §3.1].

Playout cap randomization. Following Wu [51, §3.1], each move is a *full* search (cap N) with probability p , and otherwise a *fast* search (cap $n < N$). The Gofer defaults are $p = 0.20$, $N = 200$ and $n = 50$; KataGo’s main run used $p = 0.25$ and $(N, n) = (600, 100)$, later raised to $(1000, 200)$. The configurations in section 7 change these values per deployment.

Technique	KataGo paper	Gofer v3	Gofer v4
Playout cap randomization	$p=0.25$, fast turns discarded	$p=0.20$, discarded	$\sim p=0.20$; fast turns kept, policy loss masked (§5.2)
Forced playouts + target pruning	✓	$\sim$ simplified	$\sim$ unchanged: forced root visits; children with <2 visits dropped from target (§4.1)
Global pooling bias	✓ (mean, scaled mean, max)	×	$\sim$ mean + max; gpool nets only
Auxiliary opponent policy	✓ $w=0.15$	×	✓ $w=0.15$, gpool nets
Ownership target	✓ cross-entropy	$\sim$ MSE, absolute frame	$\sim$ MSE on tanh, side-to-move frame
Score target	pdf + cdf losses	×	$\sim$ Huber on margin/20
Go-specific input features	liberties, ladders, ...	×	×
Rules / board-size randomization	✓	×	×
Symmetry augmentation	—	×	new: per-sample D4
Replay window	power law, eq. (3)	FIFO 50k rows	✓ power law + recency weighting
Validation split	—	by row (leaks)	new: by game
Weight averaging	EMA of snapshots	×	$\sim$ per-step EMA, 0.999
LR schedule	fixed, reduced early/late	constant	$\sim$ warmup + cosine per cycle
Architecture change	train next size in parallel	re-bootstrap	new: distillation from champion
Gating	$\geq 100/200$ wins	0.55 + Wilson, ≤ 200 games, interim stops	new: SPRT + Wilson fallback; fewer false promotions
Preemption safety	—	restart cycle	new: stage + step resume
Data format	—	JSONL	new: binary shards, $\sim 40\times$ smaller

Table 1: Technique-by-technique comparison. “—” means the KataGo paper does not describe the item. The v4 column covers only what is implemented and tested. None of the v4 differences has been tested for playing strength (section 9).

Forced playouts and target pruning (simplified). Full searches force a minimum number of visits to every root child c that has been expanded:

$$n_{\text{forced}}(c) = k + \left\lfloor \sqrt{P(c)(N+1)} \right\rfloor, \quad k = 2, \quad (4)$$

where $P(c)$ is the noised prior. KataGo uses $n_{\text{forced}}(c) = (kP(c) \sum_{c'} N(c'))^{1/2}$ and forces these visits by setting the urgency of an under-visited child to infinity inside the search. Gofer instead runs the forced playouts as a separate pass, right after the root is expanded and before the main search. The policy target differs more. KataGo subtracts forced visits from each child only as long as that child would not have overtaken the best child under PUCT. Gofer simply removes children with fewer than two visits and renormalizes. This is cruder: it removes single-visit noise but keeps any extra visits that forcing added to a child that stayed bad. Both mechanisms predate v4 and are unchanged by it.

Two corrections to the search. Validating the v4 gate uncovered two defects in this search, both predating v4. Selection used a child’s mean without negating it, although node values are kept from each node’s own side to move, so the engine was choosing the move best for its opponent; and two consecutive passes were not treated as the end of the game, so a finished position was scored by the evaluator rather than by the rules. Section 8.5 gives the evidence and the measured effect. Everything described in this section is the corrected search.

Move selection and parallelism. For the first 16 plies the move is sampled in proportion to visit counts ($\tau = 1$); after that the most-visited move is played. The policy target is always the full visit distribution, whatever move is played. Games run concurrently and share one batched evaluator, so the neural network sees real batches. Each game’s random generator is seeded from the run seed and the game index only, so the output does not depend on how many games run in parallel. It can still depend on the inference backend if two backends return different network outputs; in our runs the two backends produced identical row counts from the same seeds (section 8.6). Seeds are derived by mixing the run seed with the game index rather than by adding them: consecutive seeds draw correlated openings, which costs game diversity here and, in the arena, biased results outright (section 8.5).

4.2 Labels and a common frame

Each recorded position gets the 8 binary input planes and 4 global features of section 5.1, plus these targets:

- π : the root visit distribution over the $S^2 + 1$ moves (pass is the last entry);
- $z \in \{-1, 0, +1\}$: the game result;
- s : the final area-score margin, with komi included;
- $o \in \{-1, 0, +1\}^{S^2}$: the final owner of each point;
- π_{opp} : the policy target of the *next* ply, i.e. the opponent’s reply, as in Tian and Zhu [46] and Wu [51, §3.4];
- a flag saying whether the move was a full search.

In v4, every target is expressed from the side to move, the same frame as the “own/opponent” input planes. Under area scoring every point belongs to one player or to neither, so each row satisfies an exact identity:

$$s = \sum_{i=1}^{S^2} o_i - \kappa \sigma, \quad \sigma = \begin{cases} +1 & \text{Black to move} \\ -1 & \text{White to move} \end{cases} \quad (5)$$

Here κ is the komi, which goes to White. The shard writer’s test checks eq. (5) on every row of real self-play output, and the fixture shard shared with the learner’s tests satisfies it with zero error.

Two defects in the v3 labels. Comparing the v3 JSONL fields with this frame found two errors. Neither raised an exception, and both went unnoticed in v3.

1. **Ownership frame.** v3 wrote ownership in the absolute frame (Black = +1) for every row, while value and the input planes are side-to-move. For White-to-move rows the target pointed the wrong way: the network had to recover the colour from the side-to-move plane just to learn a sign flip.
2. **Opponent policy.** The field named `policy_opp` held the policy of the *previous* ply, not the opponent’s reply. It is correlated with the right target, which is why nothing looked wrong. But the auxiliary loss it was meant to feed would have taught the network to predict the past.

v4 fixes both in the engine. The shard format stores the side-to-move ownership and the true next-ply policy. The JSONL writer keeps its old field meanings for compatibility and adds `policy_next` and `score_margin`; the learner corrects legacy rows when it loads them (section 5.1). π_{opp} is recorded only when the next ply was a full search. After a fast search the visit distribution is too noisy to imitate, and the row stores an all-zero vector, which the loss treats as missing.

Keeping fast-search rows. KataGo records only full-search positions [51, §3.1], and v3 did the same (-full-only). v4 shards keep every position and flag fast ones with `full_search = 0`. The learner then masks the two policy losses to full-search rows and trains value, ownership and score on every row (section 5.2). With $p = 0.2$ this gives about $5\times$ more rows per game for those heads, at no extra self-play cost. The benefit is smaller than that ratio suggests. Wu [51] note that value training is limited by the number of *games*, since there is one noisy result per game. Extra positions from the same game share the same z and are strongly correlated. The gain we expect is therefore mostly in the per-point ownership signal and in the variety of positions seen, not in independent value labels. The risk is overfitting to game outcomes. The learner’s game-level validation split (section 5.1) is what would reveal it. This choice has not been ablated for strength (section 9).

4.3 The shard format

When the output path ends in `.npz`, the engine writes a *Gofer shard v1*. This is a standard NumPy archive [16]: a ZIP file [38] of `.npy` arrays [25], compressed with DEFLATE [7]. There is one shard per self-play run, and every row in a shard has the same board size. Table 2 lists the arrays. Any `np.load` can read a shard, so the learner, the replay index and ad-hoc analysis all use it without a custom parser. The learner’s JSONL packer produces the same layout, so the two producers are interchangeable.

Three properties matter for the rest of the pipeline:

- *Atomic writes.* The engine writes to a temporary file and renames it at the end, so a crash never leaves a truncated shard for the trainer to read.

Array	dtype	Shape	Content
<code>spatial</code>	uint8	$[R, 8, S, S]$	binary input planes
<code>globals</code>	float32	$[R, 4]$	komi/10, move/ (S^2+1) , is-black, is-white
<code>policy</code>	float32	$[R, S^2+1]$	π , root visit distribution
<code>policy_opp</code>	float32	$[R, S^2+1]$	π_{opp} , next-ply target (zeros if absent)
<code>value</code>	float32	$[R]$	z , side to move
<code>score</code>	float32	$[R]$	s , side to move, komi included
<code>ownership</code>	int8	$[R, S^2]$	o , side to move
<code>full_search</code>	uint8	$[R]$	1 if the move was a full search
<code>game_id</code>	int32	$[R]$	game index within the shard
<code>move_num</code>	int16	$[R]$	ply number
<code>meta</code>	uint8	$[K]$	UTF-8 JSON: format, version, board size, rows, games, komi, seed, generating model (ONNX hash), commit

Table 2: Arrays in a Gofer shard v1 with R rows on an $S \times S$ board. Every target uses the side-to-move frame.

- *Immutability.* No later stage rewrites a shard, so the replay buffer is just a choice of which shards to read (section 6.4), and copying a run directory between machines is incremental.
- *Provenance.* The `meta` array records which network generated the games (the first 8 bytes of its ONNX SHA-256, or `heuristic`). The orchestrator can read it without decompressing any data array.

Size. We measured size on the same four self-play games (seed 1, 20 playouts, 327 positions) written in both formats. JSONL [3] took 779,565 bytes (2,384 bytes per row), and the shard took 18,969 bytes (58 bytes per row), $41\times$ smaller. Uncompressed, a shard row is 1,416 bytes, dominated by the 648 bytes of input planes and the two 328-byte policy vectors. DEFLATE does the rest: the planes are binary and sparse, and most policy entries are exactly zero. The learner’s packer, run on a larger legacy file (section 8), gives a similar $38\times$. Loading a shard is one decompression per array, with no per-number text parsing.

5 The Learner

The learner is the Python package `gofer_train`, built on PyTorch [37]. It reads shards, trains one candidate per cycle, and exports it. The v3 command-line tools keep their names and flags and now call into it.

5.1 Data path

Loading. A source may be a legacy JSONL file, one shard, or a directory of shards. Directories are ordered oldest to newest by modification time. Given a window of W rows (from the orchestrator, eq. (3)), the loader walks the sources from newest to oldest, stops once W rows are collected, and keeps exactly the newest W . Old shards are never decompressed. Game identifiers are re-based per source, so games stay distinct across shards.

Legacy JSONL. Old data is converted into shard conventions on load:

- Ownership, stored with Black = +1, is multiplied by -1 on White-to-move rows.

- The field named `policy_opp` is dropped, because in v3 it held the policy of the move *before* the current position. The correct reply target `policy_next`, added to JSONL in v4, is used when present.
- Game boundaries are recovered from resets of the move number, since v3 recorded no game id.

A test compares converted ownership with the raw file for every row.

Device residency. The window fits comfortably in accelerator memory. 50,000 9×9 rows take $50,000 \times 8 \times 81$ bytes ≈ 32 MB of `uint8` planes, plus float targets. So the whole window is copied to the training device once. Each step gathers a batch by index on the device and converts planes to float there. This removes the v3 `DataLoader`, which built a tensor from a Python list for every row, every epoch, and it removes host-to-device copies from the inner loop.

Game-level split. Positions from one game are strongly correlated. Silver et al. [41] sampled one position per game for exactly this reason when training AlphaGo’s value network. v3 split train and validation by row, so a validation position usually had near-duplicates in training, and validation loss partly measured memorisation. v4 assigns whole games to one side. A test checks that the two sets of game ids are disjoint. One consequence: v4 validation losses are *not comparable* with v3 numbers.

Recency sampling. With the default decay $\lambda = 0$, batches are drawn uniformly from the window, as in KataGo. With $\lambda > 0$, row i of an n -row window ($0 = \text{oldest}$) is drawn with probability proportional to $\exp(\lambda(i/n - 1))$, so the newest data is sampled e^λ times as often as the oldest. KataGo does not use this. It is offered as a knob for small runs where the window is dominated by early, weaker games. The CPU preset uses $\lambda = 1.5$ and the GPU preset $\lambda = 2$. Neither value has been tuned.

Symmetry augmentation. Each sample in a batch gets an independent random element of the dihedral group D_4 . The same element is applied to the spatial planes (all are point-local, including ko and move history), to the S^2 board entries of both policy targets (the pass entry is left alone), and to the ownership map. A test places one asymmetric stone together with its policy mass and ownership, and checks two things: under all sampled symmetries the three stay on the same point, and all eight images of the point are reached.

5.2 Loss

Let a batch B have rows i with full-search flag f_i , value target $z_i \in \{-1, 0, 1\}$, policy target π_i , reply target π_i^{opp} (possibly empty), ownership $o_i \in \{-1, 0, 1\}^{S^2}$ and final margin s_i (possibly unknown). All targets are from the side to move. Define the row sets

$$F = \{i : f_i = 1, \sum \pi_i > 0\}, \quad F^{\text{opp}} = \{i : f_i = 1, \sum \pi_i^{\text{opp}} > 0\}, \quad K = \{i : s_i \text{ known}\}.$$

With $\text{CE}(p, q) = -\sum_m p(m) \log q(m)$ and mean_X the mean over a set X (zero if X is empty), the loss is

$$\begin{aligned} L = & w_p \text{mean}_F \text{CE}(\pi_i, \hat{\pi}_i) + w_v \text{mean}_B (z_i - \hat{v}_i)^2 + w_o \text{mean}_B \frac{1}{S^2} \|o_i - \hat{o}_i\|^2 \\ & + w_{\text{opp}} \text{mean}_{F^{\text{opp}}} \text{CE}(\pi_i^{\text{opp}}, \hat{\pi}_i^{\text{opp}}) + w_s \text{mean}_K H_1(\hat{s}_i - s_i/20) + w_d L_{\text{distill}}, \end{aligned} \quad (6)$$

where H_1 is the Huber loss [21] with threshold 1, and $\hat{v}, \hat{o}$ are tanh outputs. The default weights are $w_p = 1$, $w_v = 1.5$ (matching KataGo’s c_g), $w_o = 0.15$, $w_{\text{opp}} = 0.15$ (matching KataGo), $w_s = 0.05$, and $w_d = 0$ unless a teacher is given. The auxiliary terms act as multi-task regularisers [4] for the shared trunk. Weight decay is decoupled from the loss [29] and applied only to convolution and linear weights, never to biases or batch-norm parameters [22].

How this differs from KataGo.

- **Value.** Like AlphaGo Zero [42] and unlike KataGo’s win/loss cross-entropy, the value is a squared error on a tanh output. The engine’s ONNX contract is a single scalar in $[-1, 1]$, and we kept it.
- **Ownership and score.** KataGo uses cross-entropy for ownership and a pdf/cdf pair over a discretised score distribution. We use regression: squared error for ownership and Huber for score. This is simpler, but it loses the score-distribution information KataGo exploits.

Both are simplifications, not claimed improvements.

Masking fast-search rows: a deliberate change with a known risk. KataGo records only full-search turns. v4 records every turn and masks only the policy term (sets F , F^{opp}). The argument for this is that the outcome, ownership and final score of a fast-search turn are exactly as valid as those of a full-search turn, because the game produces them rather than the search. With $p = 0.20$, each game then yields about $1/p = 5\times$ more value, ownership and score rows at no extra self-play cost.

The argument against it is the correlation that Silver et al. [41] warned about. Extra rows from the *same* games add little independent information about z , and they can overfit the value head to individual games. KataGo’s choice may have been made for exactly this reason.

We therefore treat masking as a hypothesis, not a result. It needs the ablation in section 10: identical runs with fast rows kept and discarded, compared in Elo per unit of self-play compute. Game-level validation (section 5.1) is what makes the overfitting risk visible in the first place. A row-level split would hide it.

5.3 Optimisation

Each cycle warm-starts from the champion’s weights (`-init-from`) with a fresh optimiser. The optimiser options are:

- **Optimiser.** SGD with Nesterov momentum 0.9 [44], or AdamW [29].
- **Schedule.** Linear warmup [15] over 5% of the steps (at most 500), then cosine decay [28] to 5% of the peak rate. This differs from KataGo’s continuous fixed rate, because v4 (like v3) trains a fresh fine-tuning run each cycle. Section 9 discusses that choice.
- **Stability.** Gradients are clipped at norm 5.
- **Mixed precision.** On CUDA the trainer uses bf16 autocast where supported, or fp16 with loss scaling [31]. `torch.compile` and channels-last layout are opt-in and limited to CUDA. On the Windows CPU we tested, Inductor compilation did not finish in 10 minutes.

Exponential moving average. The trainer keeps a shadow copy $\bar{\theta}$ of all parameters and batch-norm statistics, updated after every step:

$$\bar{\theta} \leftarrow d_t \bar{\theta} + (1 - d_t) \theta, \quad d_t = \min\left(d, \frac{1 + t}{10 + t}\right), \quad d = 0.999. \quad (7)$$

Evaluation, best-checkpoint selection and export all use $\bar{\theta}$. This is iterate averaging in the sense of Polyak and Juditsky [39]. It plays the same role as KataGo’s EMA of weight snapshots (decay 0.75 over snapshots taken every 250k samples) and relates to SWA [23]. Our averaging is per step and much shorter (horizon $\approx 1/(1 - d) = 1000$ steps), which suits cycles of a few thousand steps. The warm-up term keeps random early weights from lingering in a short run. The update is one fused `_foreach_lerp_` over all floating-point tensors.

Checkpoints and stopping. Validation runs every epoch or every k steps, and early stopping has a patience counted in evaluations. Every write is atomic (temporary file, then rename):

- `best.pt` holds the best EMA weights as a plain state dict, so every v3 consumer still works.
- `best.ckpt` adds the architecture and the metrics.
- `last.pt` holds the full state: weights, EMA, optimiser, scheduler, loss scaler and counters.

`-continue` restores that full state and resumes at the same step. A preempted spot instance therefore loses at most `ckpt_every` steps of training (section 7).

5.4 Architectures

Two families share one interface. Both return policy logits, value and ownership, and the newer one also returns score and reply-policy outputs.

Legacy. This is the v3 network, bit-compatible with the champion’s state dict:

- A convolutional stem and b post-activation residual blocks with c channels.
- The flattened trunk is concatenated with an embedding of the global inputs.
- Fully connected policy and value heads read that vector, plus a 1×1 ownership convolution.

The fully connected heads dominate the model. For 6×64 they hold 766,484 of its 1,215,444 parameters (63%). Their size also grows with S^2 , which ties the network to one board size.

gpool. This family follows KataGo’s design:

- **Trunk.** Pre-activation residual blocks [17]. The global inputs enter as a per-channel bias on the stem.
- **Global pooling.** Every third block routes c_g of its channels after the first convolution through BN, ReLU and mean-and-max pooling, then a linear layer whose output biases the remaining $c - c_g$ channels. This is KataGo’s global-pooling bias, without the board-width-scaled mean. That term is a constant multiple of the mean on a fixed 9×9 board and would need to be added back for mixed sizes.

- **Initialisation.** The second convolution of each block starts at zero, so every block begins as the identity. This is similar in spirit to Goyal et al. [15]’s zero- γ initialisation. It targets the seed sensitivity recorded in the repository’s ADR 0005, where one 6×32 seed diverged. We have not re-run that seed study.
- **Policy head.** A 1×1 convolution with a pooled bias produces two maps: the move policy and the opponent-reply policy. Two pass logits come from the pooled features.
- **Value head.** It pools a 1×1 convolution and feeds a two-layer MLP producing value (tanh) and score. Ownership comes from the value head’s spatial features, as in KataGo.

No layer depends on S , so the parameter count is the same for every board size. A test runs one network on 9×9 , 13×13 and 19×19 inputs.

Table 3 lists the presets. One caveat matters. `gpool-6 $\times$ 64` has $2.7\times$ fewer parameters than legacy 6×64 , because the fully connected heads are gone, but it needs about the *same* multiply-accumulates per position. Its CPU latency is no lower (section 8). The parameter saving is not a speed saving.

Preset	Params	Head params	MACs / position	ONNX size
legacy-6 $\times$ 64 (v3 champion)	1,215,444	766,484	37.0 M	4.9 MB
legacy-4 $\times$ 48	682,356	511,908	14.2 M	2.7 MB
<code>gpool-6$\times$64</code>	446,631	12,935	35.2 M	1.3 MB
<code>gpool-10$\times$96</code>	1,634,687	16,007	130.7 M	4.1 MB
<code>gpool-15$\times$128</code>	4,321,575	33,223	345.7 M	10.2 MB

Table 3: Architecture presets on 9×9 . “Head params” counts every parameter outside the stem and trunk. MACs count convolution and linear layers only, measured with forward hooks on a single position. ONNX sizes are for policy+value exports.

5.5 Changing architecture by distillation

KataGo grows its network during a run by training the next size concurrently on the same data and switching once its loss catches up [51]. That needs a second trainer for the overlap period. v3 could not change size at all without re-bootstrapping, estimated at 4–5 days (section 2.3).

v4 adds knowledge distillation [18] from the current champion. Any checkpoint, of any architecture, can be passed as `-teacher`. On the same augmented batch, the teacher’s outputs add

$$L_{\text{distill}} = \text{mean}_B \text{CE}(\text{softmax}(\ell_i^T), \hat{\pi}_i) + \text{mean}_B (\hat{v}_i - v_i^T)^2 \quad (8)$$

to eq. (6), with $w_d = 1$ by default.

The student therefore learns from the replay targets and from the champion’s own policy and value on every position in the window, including fast-search positions whose policy targets are masked. The added cost is one teacher forward pass per batch. The student must still pass the ordinary gate against the champion before it is used for self-play. Whether distillation actually avoids the re-bootstrap cost is an empirical question we have not answered (section 9).

5.6 Export

The exporter recovers the architecture in one of three ways:

- from `best.ckpt` or `last.pt`;
- from a `best.ckpt` stored next to a bare `best.pt`;
- by inference from tensor shapes, for pre-v4 checkpoints.

Older checkpoints without the ownership head load with that head freshly initialised. Any other missing or unexpected key is an error.

The graph is exported with ONNX opset 18 [34] and exposes only `policy_logits` and `value`, the engine’s contract. The architecture, a hash of the weights, and the training step are embedded as ONNX metadata.

Every export then runs a parity check. ONNX Runtime and PyTorch evaluate 16 random positions, and the export fails if any output differs by more than 10^{-3} . The check caught a real problem while being built. The legacy TorchScript exporter restores the *wrapper* module’s training flag recursively after export, which silently switched batch norm back to batch statistics in the reference. The reference output then differed from the ONNX graph by 1.8. The fix was to put the wrapper, not just the inner network, in evaluation mode.

`-quantize-int8` also writes a dynamically quantised copy [24] for CPU inference. It has not been gated for strength and is not used by default.

6 Orchestration

The orchestrator (`python -m training.pipeline`) replaces the v3 shell loop. It reads one typed configuration file, runs the cycle of fig. 1, and keeps all run state in a single directory. This section covers the cycle and its checkpoints, the overlap between self-play and training, the replay window, the promotion gate, and how champions are published. Section 7 covers where the loop runs.

6.1 The cycle as a checkpointed state machine

A cycle N runs six stages in order:

1. **self-play**: write shard N with the current champion, or with the heuristic evaluator before any network exists;
2. **train**: start from the champion’s weights and train on the replay window (section 6.4);
3. **export**: write the candidate ONNX model and check it for parity (section 5.6);
4. **gate**: play the candidate against the champion (section 6.5);
5. **promote**: on acceptance, copy the candidate to a new immutable generation;
6. **publish**: register and release the new champion (section 6.6).

If fewer than a configured number of rows exist (`min_rows_to_train`), the cycle records the last five stages as skipped and only accumulates data. The first network is promoted unconditionally, after a short sanity match against the heuristic, because there is no champion to gate against.

6.2 Resumability

Cheap hardware gets interrupted: spot instances are reclaimed with a short warning [1], and free CI jobs have hard time limits (section 7.4). v3 restarted the whole cycle after any failure. In v4 a restart resumes at the first unfinished stage. Five invariants make this safe:

1. *Atomic state.* `state.json`, which holds the last completed cycle, the stages done in the current cycle and the champion lineage, is written to a temporary file and renamed into place. Shards (section 4.3) and history records are written the same way.
2. *Mark after success.* A stage is recorded as done only after it finishes. Counters such as the lifetime row count change in the same write, so a crash between the work and the write cannot count rows twice.
3. *Idempotent gating.* Each arena batch writes its own report file. A resumed gate reuses finished batches and replays nothing.
4. *Mid-run training resume.* If a previous training attempt left a checkpoint, the retry passes `-continue` so the learner restores optimizer and schedule state (section 5.3).
5. *Clean termination.* `SIGTERM`, sent by `docker stop`, preemption notices and CI timeouts, is turned into an orderly shutdown that stops background self-play and inference servers.

A test injects a failure into the training stage, starts a new orchestrator process on the same directory, and checks that self-play is not repeated and that rows are counted once (section 8.6).

6.3 Overlapping self-play with training

In KataGo and in distributed reinforcement-learning systems, data generation runs continuously and apart from the learner [10, 19, 51]. A single machine with a GPU is in the same situation on a small scale: self-play is mostly CPU work, training is mostly GPU work. With `overlap_selfplay` enabled, the orchestrator starts cycle $N+1$'s self-play in the background as soon as cycle N 's self-play finishes, using the champion at that moment. The games are therefore at most one generation stale, and the stage- $N+1$ self-play step only waits for whatever is left. Overlap is off in the CPU-only configurations, because there both workloads compete for the same cores.

Prefetched games are written to a staging directory and moved into the replay directory only when their own cycle begins. Without that, the trainer of cycle N could see cycle $N+1$'s shard: the learner reads the shard directory newest-first, so a prefetch that finished early displaced the very data the cycle was meant to learn from, and whether it did so depended on wall-clock timing.

6.4 Replay as a window over immutable shards

Experience replay reuses recent data across updates [27, 32]. v3 kept one JSONL file, capped at 50,000 rows, and rewrote the whole file whenever it trimmed it. v4 never rewrites data. The buffer is the newest shards whose rows fill the current window, and the learner receives the shard directory plus the window size in rows (`-window-rows`). The window follows KataGo's power law, eq. (3), applied to the lifetime row count N_{total} . The anchor c is `min_window_rows`: 20,000 on CPU and 100,000 on a GPU, against KataGo's 250,000. The window is capped at `max_window_rows`. The orchestrator owns this value because only it knows the lifetime total, including shards that have left the live directory. Shards older than `archive_factor` times the window move to `selfplay/archive/`, which keeps the directory, and therefore any CI cache (section 7.4), bounded.

6.5 Sequential gating

Model. Under the Elo model [9], a candidate that is Δ Elo stronger than the champion scores

$$p(\Delta) = \frac{1}{1 + 10^{-\Delta/400}} \quad (9)$$

per game. On 9×9 with komi 6.5 a game cannot be drawn, so a gate is a sequential test on Bernoulli trials. In what follows, w and ℓ are the candidate’s wins and losses so far.

Previous gates. KataGo promotes a candidate that wins at least 100 of 200 games [51, App. E]. This is a non-regression check: its purpose is to keep a clearly worse network out of self-play, and at $\Delta = 0$ it accepts about half the time. The v3 gate was stricter: a win rate of at least 0.55 with a Wilson lower bound [50] above 0.5. But the Go arena also *stopped early*: after 20 games it rejects once 0.55 is out of reach, and after 100 games it accepts as soon as both conditions hold. Checking a significance condition after every game inflates the false-positive rate; this is the repeated-significance-testing problem of Armitage et al. [2]. Table 4 shows the effect: a v3 candidate with no real gain is promoted 8.0% of the time, almost three times the 2.8% of a single test at 200 games with the same bound.

SPRT. v4 uses Wald’s sequential probability ratio test [49], the procedure Stockfish’s Fishtest uses to accept engine changes [45]. It compares $H_0: \Delta = \Delta_0$ against $H_1: \Delta = \Delta_1$ using the log-likelihood ratio

$$\Lambda(w, \ell) = w \log \frac{p_1}{p_0} + \ell \log \frac{1 - p_1}{1 - p_0}, \quad p_i = p(\Delta_i), \quad (10)$$

where draws, if there were any, count as half a win and half a loss. The test accepts H_1 when $\Lambda \geq \log \frac{1-\beta}{\alpha}$ and accepts H_0 when $\Lambda \leq \log \frac{\beta}{1-\alpha}$. The defaults are $\Delta_0 = 0$, $\Delta_1 = 35$ and $\alpha = 0.05$, $\beta = 0.10$, which puts the bounds at -2.25 and $+2.89$. Two practical details:

- *Batches.* The test is checked after arena batches of 40 games, not after every game. A batch has equal numbers of games with each colour, and the engine can batch its network evaluations. Batches run with `-arena-play-all`: the engine has its own in-match promotion stop, and leaving it enabled would put a second stopping rule on the same stream this test is already reading sequentially (section 8.5).
- *Cap.* The gate plays at most 600 games by default. If the test is still undecided there, the v3 rule decides.

Exact operating characteristics. We computed each gate’s acceptance probability P_{acc} and expected number of games $E[n]$ exactly, by dynamic programming over (games played, wins). The computation calls the orchestrator’s own decision code and mirrors the Go arena’s stopping rule, so it describes the gates as they run (`paper/analysis/gate_oc.py`). A 20,000-trial Monte Carlo simulation of the same procedures agrees within sampling error. The only modelling simplification is that games finish in order, whereas the real arena plays them in parallel. Table 4 and fig. 2 show the results.

What SPRT buys, and what it costs. Three findings stand out.

- *Error control.* SPRT keeps the false-promotion rate close to its nominal level: 2.3% at $\Delta = 0$, against v3’s 8.0%. It also has more power for real gains: at $\Delta = +50$ it promotes 90% of candidates, against 67%.

True Δ Elo	KataGo 100/200		Gofer v3		v4 SPRT (default)		v4 SPRT (Actions)	
	P_{acc}	$E[n]$	P_{acc}	$E[n]$	P_{acc}	$E[n]$	P_{acc}	$E[n]$
-100	0.000	200	0.000	142	0.000	94	0.000	61
-50	0.025	200	0.001	159	0.000	150	0.000	94
-20	0.228	200	0.017	171	0.001	254	0.003	138
+0	0.528	200	0.080	175	0.023	397	0.029	176
+20	0.812	200	0.249	172	0.233	492	0.149	205
+35	0.933	200	0.452	161	0.605	444	0.349	209
+50	0.982	200	0.672	145	0.899	339	0.605	196
+100	1.000	200	0.991	105	1.000	154	0.993	116
+200	1.000	200	1.000	100	1.000	85	1.000	58

Table 4: Exact acceptance probability and expected games for four gates as a function of the candidate’s true advantage. v3 includes the Go arena’s interim stops. The v4 columns use SPRT($0, \Delta_1$) with $\alpha = \beta = 0.05$: the default configuration ($\Delta_1 = 35$, batches of 40, cap 600, $\beta = 0.10$) and the free-runner configuration of section 7.4 ($\Delta_1 = 50$, batches of 24, cap 240).

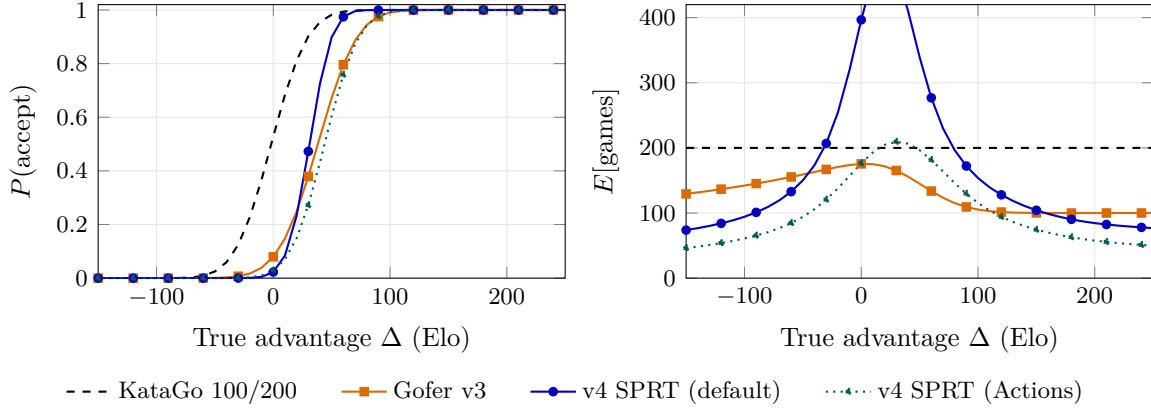


Figure 2: Operating characteristics of the four gates in table 4: the probability of promotion (left) and the expected gate length (right), against the candidate’s true advantage.

- *Not uniformly cheaper.* Clearly worse candidates are rejected sooner than in v3 (94 against 142 expected games at -100), and very strong ones are accepted sooner at $+200$ (85 against 100). But near the indifference region SPRT plays more than twice as many games (397 against 175 at $\Delta = 0$). The v4 gate is an error-control improvement, not a saving in games.
- *The cap matters.* With the default cap of 600 games, a candidate that is truly $+35$ Elo stronger is promoted 61% of the time, below the nominal $1 - \beta = 0.90$, because many such tests reach the cap undecided. Raising the cap is the main lever: at 800 games the same candidate is promoted 74% of the time, which is what the GPU configuration uses. The free-runner configuration’s cap of 240 can only detect large gains (35% at $+35$, 61% at $+50$). On a small budget, that configuration can only reliably detect large gains. This is acceptable early in a run, when each generation typically gains a lot, but it should be revisited once gains become small.

Table 4 also shows what KataGo’s gate is for. It accepts 23% of candidates that are *20 Elo worse*. That is consistent with its stated purpose: a cheap guard against large regressions in a run where most candidates improve.

6.6 Promotion, the Elo ladder and publishing

Generations. A promoted candidate becomes generation g , stored as three immutable files: the ONNX model, the weights, and the full checkpoint. Nothing overwrites or deletes a generation. Numbers are never reused, even after a rollback. Each generation carries a *ladder* Elo: the previous champion’s value plus the gain measured at its gate, with generation 1 at 0. The ladder is only a progress indicator. It is biased upwards because a candidate is recorded only when its measured gain was large enough to pass the gate. It is also blind to intransitivity. A proper rating would fit all cross-generation games jointly, as KataGo did with BayesElo [6, 51]; we leave this for future work.

Registry and releases. The publish stage copies each new champion into a registry directory with a JSON card (gate statistics, ladder Elo, SHA-256) and appends it to an index. The index tracks the current `best` and the `previous_best`. An optional *alias*, a fixed path such as `models/gofer-9x9-best.onnx` that play and analysis commands can point at, is overwritten with each new best. The registry is where the history is kept. Optionally, each champion is also published as its own GitHub Release, so older champions remain downloadable. A failed upload is recorded and retried later; it never stops training.

Guarding against a false best. A gate can pass by chance (2.3% at $\Delta = 0$ in table 4), or a candidate can beat the current champion while losing to earlier ones. Before publishing, an optional *regression match* plays the new champion against the champion `regression_lookback` generations earlier. If the new champion scores below a threshold, it is recorded as *held*: the alias and the latest release do not move. It can also be demoted, so that self-play falls back to the previous champion. A `rollback` command re-points `best`, the alias and the latest release to any earlier generation, and can optionally make that generation the training champion again. Tests cover all of these paths (section 8.6).

7 Infrastructure

KataGo’s main run used up to 28 V100 GPUs for 19 days [51, App. C]. Gofer has no compute budget, so v4 is designed to run meaningfully on free resources first and to use a single rented GPU efficiently later. One codebase and one configuration format serve four environments: a laptop, a free CI runner, a container on any GPU host, and a spot instance chosen across clouds. This section describes each, and how the time and cost of a run are estimated before paying for it.

7.1 Inference backends

The engine evaluates networks in one of two ways. The *in-process* backend links ONNX Runtime [35] into the Go binary through cgo and loads the pinned shared library (version 1.26.0) at run time. The *sidecar* backend sends batches over HTTP to a Python inference server. The in-process backend is the default: in the end-to-end run it finished the same 20-game game in 1.6s instead of 8.9s (section 8.6). The sidecar needs no C toolchain, which makes it the choice for quick local tests and for CI runners without cgo. The export step checks PyTorch against Python ONNX Runtime (section 5.6). A separate, pre-existing harness checks that the Go in-process runtime reproduces the Python runtime’s outputs. The orchestrator downloads the pinned ONNX Runtime build itself on Linux and Windows x86-64.

7.2 Container image

A multi-stage Docker [30] build compiles the engine with the ONNX Runtime tag in a Go image, then copies the binary into a PyTorch runtime image together with the pinned ONNX Runtime library, the learner and the orchestrator. The same file builds a CUDA variant for GPU hosts and a slim CPU variant, selected by a build argument. A workflow publishes both to the GitHub container registry, which is free for public repositories, so a rented machine only has to pull an image. The run directory is a mounted volume. Stopping the container sends `SIGTERM`, which the orchestrator handles cleanly (section 6.2). With a restart policy, the container resumes the run after a crash or reboot.

7.3 Rented and preemptible GPUs

For cloud GPUs, a SkyPilot [53] task file describes the job independently of the provider. SkyPilot provisions the cheapest acceptable GPU among the clouds the user has credentials for, preferring spot capacity. Its managed-job mode relaunches the task after a preemption. The run directory is mounted from an object-storage bucket, so a replacement machine starts with the state that the preempted one left, and the orchestrator resumes at the first unfinished stage (section 6.2). The learner’s periodic checkpoints (section 5.3) bound the training work lost to one preemption. Two smaller tools cover the remaining cases: a bootstrap script prepares a plain Linux virtual machine (Go, ONNX Runtime, a CUDA or CPU PyTorch, engine build and self-test), and an `rsync` wrapper copies a run directory to or from any SSH host. Because shards and generations never change, repeated copies only transfer new files.

7.4 Free-tier training on GitHub Actions

GitHub-hosted runners are free for public repositories [12]. A standard Linux runner has 4 vCPUs and 16 GB of memory, and a job may run for at most 6 hours [14]. v4 uses this as a no-cost training backend:

1. A scheduled workflow starts every 6 hours. It is opt-in: it only runs after a repository variable is set.
2. The job restores the run directory from the Actions cache, trains, and saves the directory back. The cache evicts entries that have not been accessed for 7 days [13]; on a 6-hour schedule that does not happen while training is enabled.
3. The orchestrator stops starting new cycles after 4.75 hours. A hard timeout of 320 minutes stops a long stage early; the per-stage checkpoints of section 6.2 mean the next job continues it.
4. Each new champion is published as a GitHub Release (section 6.6), which is permanent. The cache is only working storage.

At most 4×320 minutes of runner time per day gives about 21 runner-hours, or about 85 vCPU-hours, per day, with no GPU. We do not claim this is fast. It is a way to make steady progress on a small network at zero cost, and to keep the whole loop exercised continuously. The gating configuration for this environment is necessarily weak (section 6.5): with a 240-game cap it promotes a +50 Elo candidate 61% of the time and a +75 Elo one 91%, but has little chance against smaller gains.

7.5 Estimating time and cost

Every stage writes its duration to an event log. The `estimate` command averages the per-stage times over the most recent trained cycles and projects a run of C cycles:

$$T(C) = C \sum_{s \in \text{stages}} \bar{t}_s, \quad \text{cost}(C) = T(C) \cdot r, \quad (11)$$

where r is the hourly price of the instance. With overlap enabled, $\bar{t}_{\text{self-play}}$ counts only the time the cycle actually waited for self-play, so T is wall-clock time and not the sum of all work. The intended workflow is to run a few cycles of the target configuration, read the projection, and then set `deadline_hours` as a hard budget. An HTML report shows the ladder Elo, each gate’s score with its confidence interval, the time spent per stage in each cycle, and a table of all cycles.

7.6 Continuous integration

A CI workflow runs on every change to the engine, learner, orchestrator, configurations or infrastructure. It runs the Go shard tests and all Python tests, a real two-cycle pipeline run on CPU with the smoke configuration, and a build of the CPU container image followed by a self-test inside it. The workflow uploads the run directory and report as artifacts. At the time of writing, the CI, free-training and image workflows have been written and syntax-checked but not yet run on GitHub; the container image has not been built (section 9).

8 Evaluation

This section reports what can be measured without a training run at scale. It covers correctness (tests and export parity), data cost, model cost, learner throughput, and, in section 8.6, end-to-end cycles of the full pipeline. None of these numbers is a statement about playing strength.

Setup. Unless stated otherwise, measurements used one Windows 11 workstation:

- Intel Core i7-1185G7 (4 cores, 8 logical processors), 16 GB RAM, no GPU;
- Python 3.13.7, PyTorch 2.12.1 (CPU build) [37], ONNX Runtime 1.26.0 [35].

A second agent session sometimes ran builds and tests on the same machine at the same time, so the timings carry noise. We report medians and treat differences under about 20% as noise.

8.1 Correctness

Tests. The learner has 27 automated tests: 19 unit tests, 6 trainer and command-line tests, and 2 export tests. They cover every change described in section 5:

- **Data.** Shard round-trips, including against the Go-written fixture, plus window selection keeping exactly the newest rows.
- **Labels.** The ownership frame of converted JSONL, the disjoint game-level split, and alignment of board, policy and ownership under all eight symmetries.
- **Loss.** The masked policy loss is zero on fast-search rows while the value loss is not.
- **Architectures.** Recovery from a bare state dict for every preset, the `gpool` network running on 9×9 , 13×13 and 19×19 , and tolerant loading of checkpoints without an ownership head.
- **Training and export.** Warm-start weight equality, `-continue` resuming at the saved step, distillation from a legacy teacher into a `gpool` student, and export of a trained `gpool` checkpoint with metadata.

One test checks a property of the Go writer: in every row of the committed Go-written fixture shard, the side-to-move score equals the sum of the ownership map minus komi for Black to move, and plus komi for White. This holds exactly, which confirms that the Go writer and the Python reader agree on the side-to-move frame. The orchestrator and Go test counts are reported in section 8.6.

Export parity. Across 16 random positions, the largest absolute difference between PyTorch and ONNX Runtime outputs was 2.4×10^{-7} for a trained `gpool` checkpoint and 2.9×10^{-6} for a legacy 4×32 checkpoint from the v3 era. Its architecture was inferred from tensor shapes, and it lacked an ownership head. Both are far below the 10^{-3} failure threshold. The int8 export loaded and ran, but it is not parity-checked, since quantisation changes outputs by design.

8.2 Data cost

We packed the repository’s committed self-play file (`training/data/samples.jsonl`: 1,722 positions from heuristic self-play) into a compressed shard with the learner’s packer:

- **Size.** The file shrank from 4,459,476 bytes to 116,935 bytes, a factor of $38.1\times$. The factor for Go-written shards is given in section 4.
- **Loading.** The shard loaded in 0.04–0.07 s over four runs. The v3 dataset loaded the JSONL in 0.29 s, but it deferred tensor construction to every epoch. The v4 JSONL converter takes 0.9 s, because it also normalises labels and recovers game boundaries.

At the v3 replay cap of 50,000 rows, the same ratio puts the replay at roughly 3.4 MB instead of 130 MB. That difference decides whether the replay fits in a free CI cache and syncs quickly off a preemptible instance.

8.3 Model cost

Table 5 gives single-threaded ONNX Runtime latency for each preset at batch sizes 1 and 16, using freshly initialised weights (latency does not depend on the weights). Three points stand out:

- **gpool-6×64** is not faster than legacy 6×64 despite having $2.7\times$ fewer parameters, consistent with their near-equal MAC counts (table 3). Its extra pooling and slicing operations cost a little at batch 1.
- legacy-4×48 is about $2\times$ faster, matching ADR 0005’s earlier measurement.
- **gpool-10×96** has $3.5\times$ the MACs of 6×64 but only about $1.5\times$ the batch-1 latency, which suggests fixed per-call overheads dominate small networks on CPU.

Self-play throughput on CPU scales roughly with the inverse of these latencies. That is why the CPU preset keeps the legacy lineage and the **gpool-10×96** preset is reserved for GPU runs.

Preset	batch 1 (ms)		batch 16 (ms)	
	median	p10	median	p10
legacy-6×64	2.83	2.49	34.4	27.2
legacy-4×48	1.30	0.87	15.2	12.1
gpool-6×64	3.34	2.70	41.1	17.8
gpool-10×96	4.18	3.61	65.1	57.6
gpool-15×128	10.07	9.03	147.2	138.1

Table 5: ONNX Runtime CPU latency with one intra-op thread and one inter-op thread, over 300 (batch 1) or 100 (batch 16) timed runs after 20 warm-up runs. The large spread for **gpool-6×64** at batch 16 reflects machine noise from concurrent jobs.

8.4 Learner throughput

`python -m gofer_train.bench` trains for 60 steps at batch 64 on the same data in two ways: the v3 loop (`DataLoader` over the JSONL dataset, legacy 6×64) and the v4 trainer (device-resident data, augmentation, EMA, gradient clipping). Table 6 gives the median and range over three runs on a lightly loaded machine. On this CPU both are compute-bound at the same rate. The v4 trainer’s 9% edge on legacy 6×64 is inside our noise threshold: earlier runs taken while the other session was busy measured both trainers at under half these rates. The v4 per-sample overheads (symmetry gather, EMA update) are therefore not visible. This is a parity result, not a speed-up. The design removes per-row Python work and host-to-device copies, which should matter on a GPU, where the network is no longer the bottleneck, but we have not measured a GPU.

Trainer	Network	Samples / s (median of 3)
v3 loop (<code>DataLoader</code>)	legacy-6×64	397 (395–435)
v4 trainer (D4 + EMA)	legacy-6×64	432 (422–434)
v4 trainer (D4 + EMA)	gpool-6×64	385 (384–390)

Table 6: CPU training throughput at batch 64 over 60 steps (`gofer_train.bench`); median (min–max) of three runs.

8.5 What the pipeline found

The first thing the v4 gate did was refuse to work. Running it on real networks, every net-versus-net game was won by White in 12–32 moves, at every playout count we tried. Because the arena alternates colours, the candidate’s score was then mechanically 0.500 and no gate could ever decide anything.

Chasing that produced eight defects, all older than v4 and all invisible to the test suite (table 7); a later reading of the engine added two more (section 8.5). They fall into two groups. Five are in the search: they made the engine play badly and, worse, made the data it generated worthless. Three are in the measurement apparatus, and these are not a separate category of bug so much as the reason the other five were survivable: each one supplied a plausible explanation that made the next one invisible. We give them in the order the evidence forced them out, because that order is the finding. After each of the first three fixes the symptom was still there.

Defect	Consequence
<i>In the search</i>	
1 Child values not negated in selection	The engine chose the move best for its opponent; more search made it weaker
2 Two passes not treated as terminal	Finished positions scored by the network, so the search walked into lost endings
3 Node pointer invalidated by reallocation	On 9×9 the root never counted as expanded: no child was ever visited
4 First-play urgency a flat constant	Every playout went down one line; the policy target collapsed to one move
5 Forced playouts floored on every child	$82 \text{ children} \times k$ outnumbered the search budget and flattened the policy target
<i>In the measurement</i>	
6 Per-game seeds correlated with the colour swap	Role attribution biased: identical players split 126–74
7 <code>-arena-enhanced baseline</code> resolved by name	The reference baseline silently enhanced both players, masking 3 and 4
8 In-match stop fired on the statistic under test	A 200-game measurement played 169; a second stopping rule ran inside every gate batch

Table 7: The defects found while building and validating the gate. None was detectable by the test suite as it stood: the arena-scale tests skip in short mode, which is the mode continuous integration runs, and the unit tests use boards small enough that defect 3 cannot occur.

1. The search preferred the opponent’s best move. Node values are stored from each node’s own side to move: the backup negates the value at every level, and evaluators return the value from the side to move. A child’s mean is therefore the *opponent’s* view, and selection must negate it. It did not, so the search maximised the opponent’s value and more search made the engine weaker.

2. The search could not see the end of the game. Two consecutive passes end a game in every game loop, but the search did not treat them as terminal, and scored finished positions with the evaluator instead of the rules. Passing into a pass therefore looked ordinary, and with komi the engine walked into lost endings. We added the two-pass rule and exact terminal scoring, and kept terminal results out of the transposition table, whose key covers stones but not pass history.

3. The arena favoured one role. With both engine fixes in, a 200-game match between two evaluators that are *literally the same code* still split 126–74 by role. Per-game seeds were linear in the game index (`seed + 2i`, `seed + 2i + 1`, and `seed + (i+1)k` for the opening sampler), while the colour swap alternates on the same parity $i \bmod 2$. Neighbouring seeds draw correlated openings, so one parity class—which is exactly one role—was systematically decided. Measured with identical evaluators over 120 games, one parity class won 100% of its games as White, and *which* class flipped with the seed:

Seed	White wins by parity class		Role split (baseline–challenger)
	unswapped	swapped	
99	51/51 (100%)	36/49	36–64
7	26/41	40/40 (100%)	55–26
2024	30/42	43/43 (100%)	55–30
<i>after deriving each seed with a mixing function</i>			
99	46/53	48/55	55–53
7	55/60	46/60	51–69
2024	46/59	42/57	55–61

This mattered more than the numbers suggest: the gate attributes wins by role, so a biased role split is a biased promotion decision. Self-play used the same consecutive seeds and now uses the same mixing.

4. On 9×9 , the search never left the root. The three fixes above did not lengthen the games, so the symptom was still unexplained. The arena’s node storage was one slice, and its accessor returned a pointer into it. Expanding a node took that pointer, appended the children—which reallocates the slice once they outgrow its capacity—and then wrote the “expanded” flag through the pointer, into the array that had just been discarded. The initial capacity was 64 nodes and a 9×9 root has 82 children, so on every 9×9 search the root stayed unexpanded: each playout broke out of the descent immediately, the transposition table answered all the later ones with the root’s own value, and **no child was ever visited**. Move selection takes the most-visited child, all of them zero, so the engine played the first legal move no matter how long it searched. Every test in the suite uses a board small enough that the reallocation never happens.

Raising the initial capacity would only move the threshold at which the mistake reappears, so the storage is now fixed-size chunks that are allocated once and never resized: a pointer stays valid for the life of the arena, and the class of error is gone rather than postponed. Search time is unchanged.

5. All the search went down one line. With the root finally expanded, 200 playouts put 100% of their visits on a single move. First-play urgency was a flat -0.2 in the child’s frame, far below a typical node value, so the first child to be visited could never be outscored. The visit distribution *is* the policy training target, so this would have trained every network on one-hot targets. Urgency is now the node’s own value minus a reduction that grows with the explored prior, as in Wu [51].

6. Forced playouts flooded the root. Wu [51, §3.2] forces extra playouts on “each child of the root that has received any playouts”, so a move the search never looked at gets none. Gofer forced a floor of k on *every* child, and did it before the main search rather than after. On 9×9 that is 82 children, so at a 200-playout budget the forced pass outnumbered the search itself. It flattened

the visit distribution, and it made the policy-target pruning a no-op, because every child had been forced above the two-visit threshold that pruning uses. Restricting forcing to visited children, after the search, sharpens the targets measurably: over 154 full-search positions at 200 playouts, the top move went from 0.159 to 0.199 of the visits, the number of moves with any visits from 41.5 to 24.3, and the entropy from 2.98 to 2.60 nats, against 4.41 for a uniform distribution over 82 moves.

The same fix also returns the compute the flood was consuming. Because the forced pass cost roughly $(\text{children} \times k)$ playouts per move—about 164 on 9×9 —regardless of the search budget, the saving is largest where the budget is smallest. Measured on self-play with every move at the full cap, one seed, cost per move:

Playouts per move	before		after	
6	53.3 ms	16.1 ms	3.3×	
50	84.8 ms	31.0 ms	2.7×	
200	171.4 ms	95.3 ms	1.8×	

At 200 playouts that matches what the arithmetic predicts, since 164 of 364 playouts were forced. At 6 playouts the arithmetic predicts far more than the measured 3.3×, because per-move costs that do not scale with the budget—building features, generating legal moves, cloning the board—dominate there, and because the repaired search descends deeper, so each remaining playout does more work. This is a saving on work that should never have been done, not a faster implementation of the same work.

7. The reference baseline enhanced both players. Forced-playout mode (`-arena-enhanced`) was resolved by comparing evaluator *names*: the enhanced side was whichever engine’s name matched the baseline’s. The project’s reproducible baseline command passes the same evaluator name on both sides, so both engines were enhanced and `baseline` silently meant `both`. Because forced playouts act at the root, which is where defects 3 and 4 lived, this masked them in precisely the measurement meant to detect a strength change: re-running that command after repairing the search moved its win counts not at all. The mode now follows the baseline *role* across the colour swap. The difference is not subtle, at 600 versus 200 playouts over 60 games:

<code>-arena-enhanced</code>	Black wins	median game
<code>none</code>	67%	76 moves
<code>baseline</code>	62%	41 moves
<code>both</code>	70%	21 moves

The baseline command now uses `none`, so it measures search scaling rather than forced-playout behaviour.

8. Measurement runs stopped on the statistic they measured. A match ends as soon as its promotion gate is decided. The accept branch is guarded by a minimum game count, which tests raise to disable it, but the reject branch is not, so a 200-game bias measurement silently played 169 games—stopping exactly when the challenger fell behind. That is optional stopping correlated with the quantity under test, the same error class this report attributes to the v3 gate (section 6.5), here inside the harness rather than the gate. The same rule ran inside every batch of the v4 gate, putting a second stopping rule on the stream the SPRT was already testing sequentially. Batches now play out in full. Re-deriving the gate’s operating characteristics without the inner stop moves them very little—false promotions at zero true gain from 2.4% to 2.3%—which is worth stating plainly: it was a correctness problem, not a large bias.

What the fixes are worth. The same experiment before and after, with identical settings—40 games, one seed, identical heuristic evaluators on both sides, a symmetric configuration, and Black searching 600 playouts against White’s 200—gives 0/40 before and 24/40 after, with the median game growing from 28 moves to 80.

The project’s own reproducible baseline (`make reproduce-9x9-baseline`: 200 games, the same matchup) makes a sharper record, because it was run at four points in this work and its answer moved each time:

Engine state	Black	White	win rate	Elo	median game
original, as committed	7	193	0.035	−576	11 moves
after defects 1–2	133	67	0.665	+119	~20 moves
after defects 3–5	133	67	0.665	+119	21 moves
after defects 6–8 (current)	149	51	0.745	+186	76 moves

Table 8: The same 200-game command, run as each group of defects came off. Elo is $-400 \log_{10}(1/p - 1)$ on the Black win rate. The second run’s file was overwritten by the third, so its median is from the summary recorded at the time.

Three things in that table are worth stating rather than leaving to the reader.

First, the answer to “is three times the search worth anything” moved from −576 to +186 Elo. That is not an error of magnitude but of sign: the committed reference baseline asserted that tripling the search was catastrophically harmful, and that file sat in the repository as the project’s strength record.

Second, rows two and three are identical—133/67, the same Elo, means of 20.0 and 19.6 moves, the same maximum of 33—although every individual game record differs between them. That is the measurement demonstrating that it was not measuring what had just been repaired: **-arena-enhanced** was silently enhancing both sides (defect 7), and forced root playouts were deciding the moves either way.

Third, the cleanest signal of the regime change is not the win rate but the game lengths: *the longest game in either middle run, 33 moves, is shorter than the median game of the final run, 76*. The final run’s distribution is 8 games under 20 moves, 39 from 20 to 49, 62 from 50 to 79 and 91 at 80 or more, against an original mean of 12.9. Those are not two samples from a shifted distribution; they are two different processes. Throughout, Black concedes komi 6.5, which at these playout counts favours White (section 8.5), so +186 Elo is measured against a handicap and understates the search advantage.

Two further checks: with identical evaluators at fair komi, 200 games split 99–101 by colour, and a 200-playout search spreads its visits over many moves instead of one.

Fair komi moved with the fix. Two tests in the repository encoded a “fair komi” of −1 for equal heuristic engines, with a comment explaining that tournament komi was not fair at test playout counts. That constant had been fitted to a search that stopped at the root: games ended near-empty and komi alone decided them. Measured again on the repaired engine, at 50 playouts over 100–150 games per point, Black scores 58% at komi 0, 51% at 0.5, 46% at 1.0, 42% at 1.5, 34% at 3.5 and 29% at 6.5. Fair komi for these engines is near 0.5, where 200 games split 99–101, and the tests now use that. The value moved twice during this work, once per round of engine fixes, which is a reminder that such a constant is a measurement of the engine, not a property of the game.

Why nothing caught it. Two things hid these defects. The repository’s continuous integration runs `go test ./... -short`, and every arena-scale test skips itself in short mode; and the unit tests all use boards small enough that the node array never reallocates, so the defect that mattered most could not appear in them. One test did observe a symptom—it logged 9 Black wins in 188 games with identical evaluators—but it asserted only on role balance and explained the colour skew away in a comment.

Three tests now run in short mode and fail on the old code: root children must be visited at all, the visits must spread over more than a handful of moves, and selection, given two root children with equal priors and visits, must prefer the one whose subtree is worse for the opponent. All three are unit tests, instant and deterministic. We first wrote a game-playing version instead—three times the search must win a majority—and it did fail on the broken engine, reporting 4 wins in 11 decisive games; but at these playout counts the outcome is noisy enough that we would not trust it as a gate in continuous integration.

Why they were survivable. None of these defects crashed. Every one produced a number that a reasonable person could explain. A search that returns the first legal move looks like a weak engine on a small budget. A flattened visit distribution looks like a network that has not learned yet. A zero-filled ownership row looks like a neutral board. A match that stops at 169 of 200 games looks like a clean sample, because nothing reports the number it was asked for. The most expensive instance is not in the code at all: a note in the repository’s known-issues file recorded the White-heavy colour split as expected from komi and first-move advantage, which is a true statement about Go and a false explanation of that measurement. It closed the investigation for two months.

The general lesson we take from this is about which experiments are worth running early. The measurement that eventually broke the case open—more search must beat less search, with everything else held equal—is not a test of any feature. It is a test of whether the system’s most basic premise holds, and its expected answer was known in advance, so a wrong answer could not be rationalised away. A project with no compute budget can afford several such experiments; what it cannot afford is a training run whose result it has no way to interpret.

And six in the orchestrator. An adversarial review of the orchestrator, run as a separate exercise against the resume and publish paths, found six defects of its own. Four concerned failure handling: an interrupt left the trainer and the arena running, because the process wrapper never signalled the child and the code meant to stop background self-play was unreachable; a job killed mid-write left a truncated arena report, which the gate then cached forever and crashed on at every resume; a prefetched shard could enter the previous cycle’s training window, because the learner reads its shard directory newest-first, so overlap silently changed what a cycle trained on depending on wall-clock timing; and replaying an interrupted publish after a failed regression check promoted the failed generation back over its replacement. Two concerned publishing: the regression baseline picked the most recently appended matching generation rather than the highest-numbered one, which after a rollback is an older network than intended, and the seed gate’s result against the heuristic—the only evidence about generation 1—was dropped from the lineage because it was not a scalar. All six are fixed and covered by tests. None would have been caught by a passing test suite; they needed someone to ask what happens when the machine is killed at each point in the cycle.

And two more, found by reading rather than measuring. A later pass over the whole engine, made for code quality rather than correctness, turned up two defects of exactly the shape

described above. Both had been present throughout the work of this section.

The first is the clearest instance in this report of an instrument returning a plausible number. Constructing an ONNX evaluator went through four functions, two of them mutually recursive, one of which decided which model it was building by comparing the URL it had been handed against the configured second sidecar. On the sidecar path the resolved model was discarded and the primary URL substituted, which made that comparison unreachable. The consequence is that `-eval-backend sidecar` with a champion in one slot and a challenger in the other pointed both engines at the same sidecar. Every sidecar-backed gate was the champion playing itself, and returned a win rate near 0.500—which is precisely the value a correctly functioning symmetric match is supposed to produce. A wrong result that coincides with the expected result is not detectable by inspecting the result.

The second is in the transposition table. Slots were indexed by the Zobrist hash modulo the table size, but no key was stored in the slot, so any two positions agreeing in the low 16 bits returned each other’s evaluation. With 2^{16} slots a collision becomes likely after a few hundred distinct positions, and a single 9×9 search visits far more than that, so wrong values were being backed up the tree throughout. The symptom is a search that plays slightly worse than its evaluator should permit, which is indistinguishable from a weak evaluator. The record field that should have held the key instead held a `Depth` that was written as the constant 1 at all three store sites and never read as anything but non-zero.

The methodological point is that these two were not found the way the previous eight were. The eight came out of experiments with predictable answers; these came out of reading the code with no hypothesis, asking only whether each piece said what it meant. The two methods find different things. An experiment cannot see a defect whose output matches the expected output, and reading cannot tell you which of the plausible behaviours the system actually exhibits. A project that can afford neither a large test suite nor much compute gets more from alternating them than from either alone.

Consequences for this report. Every strength number produced before these fixes is void, including the committed baseline and the gate scores in section 8.6, which were measured while the search was still stopping at the root. No long training run had been done, so no trained model was lost; the self-play data generated so far carries policy targets from a search that never descended, and is not worth training on.

8.6 Pipeline end to end

Tests. The Go side has 20 tests across three files. Seven cover the shard writer: the NPY header layout, a round trip of real self-play output through the ZIP container checking eq. (5) on every row, next-ply policy labelling, and rejection of rows that would mix board sizes, carry non-finite floats or omit ownership. Seven cover the search defects of section 8.5. Six cover atomic artifact writes, including the Windows rename retry.

The orchestrator has 37 tests. They replace Go, PyTorch and ONNX Runtime with a fake executor that writes the same output files the real tools would. They cover:

- a three-cycle run with a seeded first champion, an SPRT acceptance that stops before the cap, and an SPRT rejection;
- a failure injected into training, followed by a resume that neither repeats self-play nor counts rows twice;

- reuse of cached gate batches after a restart;
- skipping training below the row threshold; hold mode; background self-play; in-process and sidcar command lines; pruning of old outputs;
- the window formula; shard archiving; configuration parsing and validation; the Elo and SPRT arithmetic, including the exact gate model of section 6.5 cross-checked against an independent Monte Carlo simulation;
- the defects of section 8.5: an interrupt must not orphan a background job, a prefetched shard must not enter the current cycle’s window, a truncated arena report must be replayed, and a replayed publish must not restore a demoted generation;
- publishing: every champion kept in the registry, the alias and latest release moving forward, a failed regression match holding back or demoting a champion, failed uploads retried, and rollback that rennumbers nothing.

End-to-end runs. We ran the real loop with the smoke configuration on the machine described in section 8 (an Intel Core i7-1185G7 with 16 GB of memory). It used the real Go engine, learner, exporter and arena, with tiny settings: 8 games per cycle, 24/8 playouts with $p = 0.25$, one or two training epochs, and gates of 20 games at 16 playouts. Table 9 lists stage times for a three-cycle run on the repaired engine. An earlier pair of runs, one per inference backend, is what the backend comparison below reports; their gate results are void (section 8.5), but the timings and the backend ratio are not affected by the search defects.

Cycle	Self-play	Train	Export	Gate	Rows	Window
1	20.8	26.9	4.7	17.5	655	559
2	0.0	20.2	8.7	12.7	661	784
3	0.0	13.0	7.2	12.9	180	839

Table 9: Wall-clock seconds per stage in a three-cycle smoke run on the repaired engine, with self-play rows produced and the replay window used for training. Self-play time is the time the cycle waited: with overlap, both later cycles’ games had already been generated in the background. Cycle 1 gates against the heuristic evaluator; later cycles use SPRT against the champion. Promotion and publishing each took under 0.2 s.

Observations.

- *Overlap.* Both later cycles waited at most 0.04 s for self-play, because their games had been produced in the background while the previous cycle trained, exported and gated. In an earlier run one cycle waited 20.9 s: overlap hides self-play only up to the length of the other stages.
- *Backend.* The same 20-game gate took 1.6 s in-process and 8.9 s over sidecars. The sidcar figure includes starting two inference servers and waiting for them to report healthy, which dominates at this scale.
- *Reproducibility.* The same seeds gave identical row counts under both backends (659 and 663), including the cycles where self-play used the network. This is consistent with the per-game seeding of section 4.1, but it is not proof of bit-identical games.

- *Gate behaviour.* The smoke networks are deliberately worthless—one or two epochs on a few hundred positions—and the gate says so. The seeded champion lost all 10 games to the heuristic evaluator, and each later candidate scored exactly 10–10 against it: with two equally weak networks and alternating colours, every game goes to whoever holds White, so the score is 0.500 by construction. The likelihood ratio stayed at -0.10 , the cap was reached and the fallback rejected. Before the search fixes this table looked the same, which is what made the defects so hard to see; the difference is that the games now vary in length (6–24 moves against the champion, up to 83 against the heuristic) instead of being 10 copies of one game.
- *Data.* Shards took 12–38 KB per cycle. They were loaded by the learner without conversion, and the parity-checked export succeeded in every cycle.

A full smoke run takes about three minutes of wall-clock time for three cycles, including the engine build.

Gate length and error rates. Table 4 in section 6.5 gives the exact operating characteristics of the v3 and v4 gates. It is derived analytically, from the implemented decision code, not measured from games. Its central results are a false-promotion rate of 2.7% for v4 against 8.0% for v3 at zero true gain, and longer v4 gates near zero gain.

Not yet measured. We have not run the container build, any GitHub workflow, a GPU run, or a free-runner training run. No network trained by v4 has been compared with a v3 champion. These gaps are listed with the other limitations in section 9.

9 Limitations

The implementations assume a small board. The shard format stores plane counts and board size dynamically, the engine takes a board size, and the `gpool` networks are size-agnostic, but two implementations are not. The shard writer and the learner’s reader both materialise a whole window in memory: at 19×19 , 100k rows of 22 planes over 361 points is roughly 800 MB in one allocation before the policy arrays. And the inference sidecar hard-codes 9×9 and eight planes, so a larger board would fail there rather than in the parts that look size-dependent. Both are ceilings rather than errors at the size we run, and both need fixing before any 19×19 work.

Platform coverage. The in-process backend needs a pinned ONNX Runtime library, which exists for Linux x86-64, Linux arm64, macOS arm64 and Windows x86-64. On anything else—an Intel Mac, for instance—the pipeline runs only through the sidecar backend, which needs just the Python wheel.

Everything measured before the engine fixes is void. The defects in section 8.5 invalidate every strength number this project had recorded, including its committed reference baseline and the gate scores from the end-to-end runs reported here. The numbers in this report were taken after the fixes, except where explicitly labelled as “before”. The fixes themselves are verified only by the measurements in section 8.5: a corrected search wins where it used to lose, and role attribution is balanced between identical players. They have not been validated by a training run.

No strength results. No v4 training run has been done at a scale that could measure playing strength. The end-to-end runs in section 8.6 check that the machinery works; they say nothing about learning efficiency. Every difference from KataGo marked $\sim$ or “new” in table 1 could help, do nothing, or hurt. In particular, we make no claim that Gofer v4 learns faster than Gofer v3, or that it is comparable to KataGo.

Untested design choices. Four choices are adopted on argument alone:

- keeping fast-search rows for the value, ownership and score heads (section 5.2), which carries a known risk of correlation and overfitting;
- recency-weighted sampling;
- per-step EMA in place of snapshot averaging;
- warm-start fine-tuning with a cosine schedule each cycle, in place of KataGo’s continuous training on one run.

The last choice has a structural cost: when a candidate is rejected, its training steps are thrown away, where a continuous learner would have kept improving. The v3 design made this choice, v4 inherits it, and it should be tested against continuous training with periodic snapshots.

Missing KataGo techniques. v4 still lacks the following:

- KataGo’s exact policy target pruning, which subtracts forced playouts using the PUCT formula (the engine only drops root children with fewer than two visits, section 4.1);
- disabling root noise on fast searches;
- the score-belief distribution (pdf and cdf losses);
- the board-width-scaled term in global pooling;
- Go-specific input features (liberties, ladders, pass-alive areas);
- randomisation of rules, komi and board size;
- handicap and branching games;
- growing the network during a run.

In Wu [51]’s ablations, removing Go-specific features and optimisations cost a $1.55\times$ training-time factor, and removing forced playouts with pruning cost $1.25\times$. v4 has only a simplified form of the latter. The `gpool` network does not depend on board size, but the feature schema, the self-play generator and the ONNX export are all still 9×9 only.

Performance. On the CPU we measured, training throughput per step is unchanged from v3 (section 8). The workload is compute-bound, and augmentation and EMA add a little overhead. The GPU-oriented features (device residency, mixed precision, compile, channels-last) have been written but not benchmarked on a GPU. The `gpool-6 $\times$ 64` network is smaller on disk but no faster to evaluate on CPU than legacy `6 $\times$ 64`.

Validation loss is only a proxy. The learner selects `best.pt` by validation loss on held-out games. Lower loss on self-play data from the current champion does not have to mean stronger play. The arena gate is the real test, and its power is limited by the number of games a free CPU runner can afford.

Unverified quantisation. The int8 export has passed only the structural checks: it loads and produces outputs of the right shape. Its effect on playing strength is unknown.

Authorship and review. The v4 code and this report were produced mostly by AI coding agents under the direction of the project owner, with tests as the main check. The report has not been externally peer-reviewed.

10 Conclusion and Future Work

Gofer v4 turns a shell-script training loop into a tested, resumable pipeline shaped by the KataGo recipe [51]:

- **Data.** Self-play writes compact shards with correctly framed labels. The learner trains on a growing window with game-level validation, symmetry augmentation, masked multi-head losses and EMA weights.
- **Architectures.** The learner supports a size-agnostic global-pooling network, and it can move to a new architecture by distilling from the champion.
- **Loop.** Candidates pass a parity-checked export and a sequential gate, and every champion is kept.

Measured against v3, the gains we can defend today are correctness, data volume, robustness, and control of false promotions at the gate (which costs longer gates for near-neutral candidates). Measured against KataGo, v4 is a partial adoption with different engineering priorities: it is built for a project with no compute budget.

The result we did not expect is section 8.5. A gate is a measuring instrument, and pointing a new one at the engine showed that its search had been selecting the opponent’s best move, could not see a game end, and was being compared in an arena that favoured one role. None of this needed a GPU to find; it needed an experiment whose outcome was predictable enough that a wrong answer was recognisable as wrong. For a project that cannot yet afford to train, that is the cheapest possible lesson: build the measurement before buying the compute, because a training run on the old engine would have burned its entire budget teaching a network to imitate a search that preferred losing moves.

Experiment protocol. The open questions are empirical, and the pipeline was built to answer them cheaply. We propose a protocol modelled on Wu’s ablations: a main run, plus runs that each change one component, compared by Elo on the pipeline’s champion ladder against cumulative self-play compute (network queries $\times$ blocks $\times$ channels², as in Wu [51]). In priority order:

1. fast-search rows kept (masked policy) versus discarded;
2. `gpool1-6 $\times$ 64` versus `legacy-6 $\times$ 64` at equal self-play compute;

3. a `gpool` student distilled from the v3 champion versus the same network trained from scratch, measured in cycles to reach the champion’s arena strength;
4. per-cycle fine-tuning versus continuous training;
5. EMA on versus off, and recency decay $\lambda \in \{0, 2\}$.

Each comparison should be repeated over several seeds. The pipeline’s cost estimator projects the price of each run from measured stage timings, so runs can be sized to the available budget before any GPU is rented.

Future engineering. Following KataGo, the next steps are:

- KataGo’s PUCT-based policy target pruning, replacing the visit threshold, and noise-free fast searches;
- a discretised score-belief head;
- liberty and ladder input features (a feature-schema version bump);
- board-size and komi randomisation in self-play, which the `gpool` trunk already accommodates.

Acknowledgments

The v4 code and this report were written with substantial help from AI coding agents (Claude, Anthropic). Two agent sessions worked in parallel: one on the learner and one on self-play data and orchestration. All numbers reported here come from commands run on the hardware described in section 8.

References

- [1] Amazon Web Services. Spot instance interruptions. <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/spot-interruptions.html>, 2026. Amazon EC2 User Guide. Accessed 2026-09-18.
- [2] Peter Armitage, C. K. McPherson, and B. C. Rowe. Repeated significance tests on accumulating data. *Journal of the Royal Statistical Society. Series A (General)*, 132(2):235–244, 1969. doi: 10.2307/2343787.
- [3] Tim Bray. The JavaScript object notation (JSON) data interchange format. Request for Comments 8259, Internet Engineering Task Force, 2017.
- [4] Rich Caruana. Multitask learning. *Machine Learning*, 28(1):41–75, 1997. doi: 10.1023/A:1007379606734.
- [5] Rémi Coulom. Efficient selectivity and backup operators in Monte-Carlo tree search. In *Computers and Games (CG 2006)*, volume 4630 of *Lecture Notes in Computer Science*, pages 72–83. Springer, 2007.
- [6] Rémi Coulom. Bayesian Elo rating. <https://www.remi-coulom.fr/Bayesian-Elo/>, 2010.

- [7] L. Peter Deutsch. DEFLATE compressed data format specification version 1.3. Request for Comments 1951, Internet Engineering Task Force, 1996.
- [8] Alan A. A. Donovan and Brian W. Kernighan. *The Go Programming Language*. Addison-Wesley, 2015.
- [9] Arpad E. Elo. *The Rating of Chessplayers, Past and Present*. Arco Publishing, New York, 1978.
- [10] Lasse Espeholt, Hubert Soyer, Rémi Munos, Karen Simonyan, Volodymyr Mnih, Tom Ward, Yotam Doron, Vlad Firoiu, Tim Harley, Iain Dunning, Shane Legg, and Koray Kavukcuoglu. IMPALA: Scalable distributed deep-RL with importance weighted actor-learner architectures. In *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 1407–1416, 2018.
- [11] Henrik Forsten. Optimal amount of visits per move. Leela Zero project issue #1416, <https://github.com/leela-zero/leela-zero/issues/1416>, 2019.
- [12] GitHub, Inc. About billing for GitHub Actions. <https://docs.github.com/en/billing/managing-billing-for-github-actions/about-billing-for-github-actions>, 2026. Standard GitHub-hosted runners are free for public repositories. Accessed 2026-09-18.
- [13] GitHub, Inc. Caching dependencies to speed up workflows. <https://docs.github.com/en/actions/using-workflows/caching-dependencies-to-speed-up-workflows>, 2026. Cache size limit and eviction of caches not accessed for 7 days. Accessed 2026-09-18.
- [14] GitHub, Inc. About GitHub-hosted runners. <https://docs.github.com/en/actions/using-github-hosted-runners/about-github-hosted-runners>, 2026. Runner hardware and the 6-hour job execution limit. Accessed 2026-09-18.
- [15] Priya Goyal, Piotr Dollár, Ross Girshick, Pieter Noordhuis, Lukasz Wesolowski, Aapo Kyrola, Andrew Tulloch, Yangqing Jia, and Kaiming He. Accurate, large minibatch SGD: Training ImageNet in 1 hour, 2017.
- [16] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant. Array programming with NumPy. *Nature*, 585(7825):357–362, 2020. doi: 10.1038/s41586-020-2649-2.
- [17] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Identity mappings in deep residual networks. In *European Conference on Computer Vision (ECCV)*, pages 630–645. Springer, 2016.
- [18] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network, 2015.
- [19] Dan Horgan, John Quan, David Budden, Gabriel Barth-Maron, Matteo Hessel, Hado van Hasselt, and David Silver. Distributed prioritized experience replay. In *International Conference on Learning Representations*, 2018.

- [20] Jie Hu, Li Shen, and Gang Sun. Squeeze-and-excitation networks. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 7132–7141, 2018.
- [21] Peter J. Huber. Robust estimation of a location parameter. *The Annals of Mathematical Statistics*, 35(1):73–101, 1964. doi: 10.1214/aoms/1177703732.
- [22] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pages 448–456, 2015.
- [23] Pavel Izmailov, Dmitrii Podoprikin, Timur Garipov, Dmitry Vetrov, and Andrew Gordon Wilson. Averaging weights leads to wider optima and better generalization. In *Conference on Uncertainty in Artificial Intelligence (UAI)*, 2018.
- [24] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2704–2713, 2018.
- [25] Robert Kern. NEP 1 — a simple file format for NumPy arrays. <https://numpy.org/neps/nep-0001-npy-format.html>, 2007. NumPy Enhancement Proposal.
- [26] Levente Kocsis and Csaba Szepesvári. Bandit based Monte-Carlo planning. In *Machine Learning: ECML 2006*, volume 4212 of *Lecture Notes in Computer Science*, pages 282–293. Springer, 2006.
- [27] Long-Ji Lin. Self-improving reactive agents based on reinforcement learning, planning and teaching. *Machine Learning*, 8(3–4):293–321, 1992. doi: 10.1007/BF00992699.
- [28] Ilya Loshchilov and Frank Hutter. SGDR: Stochastic gradient descent with warm restarts. In *International Conference on Learning Representations*, 2017.
- [29] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.
- [30] Dirk Merkel. Docker: Lightweight Linux containers for consistent development and deployment. *Linux Journal*, 2014(239), 2014.
- [31] Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. Mixed precision training. In *International Conference on Learning Representations*, 2018.
- [32] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharmashan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015. doi: 10.1038/nature14236.
- [33] Francesco Morandini, Gianluca Amato, Marco Fantozzi, Rosa Gini, Carlo Metta, and Maurizio Parton. SAI: a sensible artificial intelligence that plays with handicap and targets high scores in 9x9 Go (extended version), 2019.

- [34] ONNX Contributors. ONNX: Open neural network exchange. <https://github.com/onnx/onnx>, 2017.
- [35] ONNX Runtime developers. ONNX Runtime. <https://onnxruntime.ai>, 2021.
- [36] Gian-Carlo Pascutto et al. Leela Zero. <https://github.com/leela-zero/leela-zero>, 2019. Open-source distributed reproduction of AlphaGo Zero.
- [37] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32*, pages 8024–8035, 2019.
- [38] PKWARE Inc. APPNOTE.TXT — .ZIP file format specification, version 6.3.10. <https://pkware.cachefly.net/webdocs/casestudies/APPNOTE.TXT>, 2022.
- [39] Boris T. Polyak and Anatoli B. Juditsky. Acceleration of stochastic approximation by averaging. *SIAM Journal on Control and Optimization*, 30(4):838–855, 1992. doi: 10.1137/0330046.
- [40] Christopher D. Rosin. Multi-armed bandits with episode context. *Annals of Mathematics and Artificial Intelligence*, 61(3):203–230, 2011. doi: 10.1007/s10472-011-9258-6.
- [41] David Silver, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, et al. Mastering the game of Go with deep neural networks and tree search. *Nature*, 529(7587): 484–489, 2016. doi: 10.1038/nature16961.
- [42] David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, et al. Mastering the game of Go without human knowledge. *Nature*, 550(7676):354–359, 2017. doi: 10.1038/nature24270.
- [43] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharmashan Kumaran, Thore Graepel, et al. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science*, 362(6419):1140–1144, 2018. doi: 10.1126/science.aar6404.
- [44] Ilya Sutskever, James Martens, George Dahl, and Geoffrey Hinton. On the importance of initialization and momentum in deep learning. In *Proceedings of the 30th International Conference on Machine Learning*, volume 28 of *Proceedings of Machine Learning Research*, pages 1139–1147, 2013.
- [45] The Stockfish developers. Fishtest: Distributed testing framework for the Stockfish chess engine. <https://github.com/official-stockfish/fishtest>, 2026. Accepts engine patches with a sequential probability ratio test on Elo. Accessed 2026-09-18.
- [46] Yuandong Tian and Yan Zhu. Better computer Go player with neural network and long-term prediction. In *International Conference on Learning Representations*, 2016.
- [47] Yuandong Tian, Jerry Ma, Qucheng Gong, Shubho Sengupta, Zhuoyuan Chen, James Pinkerton, and C. Lawrence Zitnick. ELF OpenGo: An analysis and open reimplement of AlphaZero. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 6244–6253, 2019.

- [48] John Tromp. The game of Go. <http://tromp.github.io/go.html>, 2014.
- [49] Abraham Wald. Sequential tests of statistical hypotheses. *The Annals of Mathematical Statistics*, 16(2):117–186, 1945. doi: 10.1214/aoms/1177731118.
- [50] Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927. doi: 10.1080/01621459.1927.10502953.
- [51] David J. Wu. Accelerating self-play learning in Go, 2019. URL <https://arxiv.org/abs/1902.10565>. Version 5, November 2020.
- [52] Ti-Rong Wu, I-Chen Wu, Guan-Wun Chen, Ting han Wei, Tung-Yi Lai, Hung-Chun Wu, and Li-Cheng Lan. Multi-labelled value networks for computer Go. *IEEE Transactions on Games*, 10(4):378–389, 2018. doi: 10.1109/TG.2018.2852806.
- [53] Zongheng Yang, Zhanghao Wu, Michael Luo, Wei-Lin Chiang, Romil Bhardwaj, Woosuk Kwon, Siyuan Zhuang, Frank Sifei Luan, Gautam Mittal, Scott Shenker, and Ion Stoica. SkyPilot: An intercloud broker for sky computing. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 437–455, 2023.